

UNIVERSIDAD PERUANA DE
CIENCIAS APLICADAS
FACULTAD DE INGENIERÍA
COMPUTER SCIENCE PROFESSIONAL PROGRAM

Curso:

1ACC0221-2610 — Big Data

Sección: 18517

**Smart Kitchen Intelligence:
Recomendador basado en Contenido,
Filtrado
Colaborativo y Modelo Híbrido
Anti-Desperdicio**

Informe Técnico — Hito 4 (Semana 11)

*Recomendadores basados en contenido, filtrado
colaborativo implícito (ALS) y recomendación mixta*

Autores

Melgar Puertas, José Guillermo u202111660

Reyna Alvarado, Gabriel Alonso u202126097

Profesor

Alarcon Delgado, Carlos Adrian

Lima, junio de 2026

Índice general

1	Introducción y Marco del Hito 4	2
1.1	Tipo de proyecto y posición en el curso	2
1.2	Ethics and Access Note	3
2	Datos y Artefactos Reutilizados	3
3	Recomendador Basado en Contenido	4
3.1	Definición del documento por producto	4
3.2	Formulación matemática de TF-IDF	4
3.3	Perfil del household y scoring	5
4	Construcción de la Matriz R	5
4.1	Unidad de las filas: sesiones, no households	5
4.2	Encodings probados	6
4.3	Normalizaciones aplicadas	7
5	Filtrado Colaborativo	8
5.1	Similitud ítem-ítem (Semana 10)	8
5.2	ALS implícito (Semana 10) y lambda iteration	8
6	Estrategias de Cold-Start	10
6.1	Sesión cold	10
6.2	Producto cold	11
7	Recomendador Híbrido (Mixed Recommendation)	11
7.1	Ablación de pesos	12
8	Protocolo de Evaluación Offline Consolidado	13
8.1	Definición del candidate pool	13
8.2	Protocolo común	13
8.3	Comparación baseline vs sistema fuerte	14
8.4	Data alignment del híbrido	14
9	Análisis de Strong y Failure Cases	15
9.1	Strong cases (≥ 2 hits en top-5)	15
9.2	Failure cases (0 hits en top-5)	15
10	Conclusiones	16

1 Introducción y Marco del Hito 4

El presente informe documenta el **Hito 4** del proyecto *Smart Kitchen Intelligence* (SKI), correspondiente al contenido de las Semanas 9 y 10 del curso y a la entrega de la Semana 11. El hito construye, sobre el dataset consolidado en el Hito 1 y las representaciones del Hito 2, un **motor de recomendación de tres capas**: (i) basado en contenido con TF-IDF sobre el catálogo, (ii) filtrado colaborativo implícito mediante factorización ALS con barrido de regularización (λ), y (iii) un recomendador híbrido que integra ambas señales con un componente *anti-desperdicio* derivado de la proximidad de vencimiento.

El informe se estructura en torno a las seis preguntas técnicas planteadas explícitamente por el profesor para la sustentación de la Semana 11:

1. **Construcción de R**: qué unidad usamos como filas, qué encodings probamos.
2. **Normalización**: qué transformación aplicamos y por qué.
3. **Lambda iteration**: cuál estrategia usamos para regularizar ALS.
4. **Justificación**: por qué esa estrategia es la mejor para SKI.
5. **Datos que respaldan**: métricas empíricas que sostienen cada decisión.
6. **Comprensión de TF-IDF**: qué hace formalmente y por qué es adecuado.

Cada sección del informe responde a una o varias de estas preguntas de manera directa, con la métrica observada al lado de cada afirmación cuantitativa.

1.1 Tipo de proyecto y posición en el curso

Según la taxonomía declarada en el *Semester Group Assignment Brief*, el sistema construido en este hito es un **recommendation system** sobre la unidad *sesión de despensa*, no una tarea pura de ranking ni de predicción supervisada. Concretamente:

- **Recomendación**: dado un perfil (household) o un seed parcial (sesión cold), el sistema devuelve un top- N de productos candidatos. Esta es la operación primaria.
- **Segmentación que alimenta el ranking**: los 38 clusters del Hito 3 actúan como capa latente que valida la coherencia interna, pero el ranking no se condiciona sobre el cluster ID; se condiciona sobre el perfil completo del household.
- **No es predicción supervisada**: no hay etiquetas binarias de “compra” vs “no compra” a predecir; el modelo aprende a re-ranear el catálogo a partir de interacciones positivas implícitas.

1.2 Ethics and Access Note

Origen de los datos. El catálogo de productos proviene de la API pública [OpenFoodFacts](#) (licencia ODbL); los patrones de comportamiento usados para parametrizar el simulador derivan del dataset público *Instacart Online Grocery Shopping* (uso académico permitido). La capa de eventos de inventario (25,444 registros) es **sintética**, generada por `src/simulation.py` sobre la base de las distribuciones de Instacart; no contiene datos personales de usuarios reales.

Por qué se permite usarlos. Ambas fuentes son públicas y su uso académico está autorizado por sus licencias. La simulación elimina cualquier identificador personal por construcción: los `household_id` son enteros 0–9 generados por el script.

Riesgos de datos personales y mitigaciones. En el dataset entregado no hay PII (nombres, direcciones, emails). Si en una extensión futura el sistema ingiriera historiales reales de hogares, se aplicarían: (i) hashing de identificadores con salt fijo en pipeline, (ii) agregación por sesión antes de persistir, (iii) eliminación de `timestamp` exacto en favor de bin temporal de 15 minutos. Para el Hito 4 estas medidas no son necesarias porque la capa de interacción es sintética.

Limitación honesta. Al ser sintético, el dataset hereda los sesgos del simulador (distribución uniforme entre households, ventanas operacionales fijas). Las métricas obtenidas son por tanto un *piso* para la calidad del sistema sobre datos reales, no un techo.

2 Datos y Artefactos Reutilizados

El motor se construye sobre el dataset `data/processed/inventory_v1.csv` producido en el Hito 1, que contiene **25,444 eventos** de inventario distribuidos en **10 households** y **50 productos únicos** a lo largo de 13 semanas. Cada evento registra `event_type` (IN/OUT), `quantity`, `timestamp`, `expiry_date`, `nutriscore` y atributos categóricos del producto.

El cluster de comportamiento entregado en el Hito 3 (DBSCAN $\varepsilon = 2.7$, **Silhouette = 0.6549**) actúa como capa latente de segmentación para validar consistencia, pero no se usa como entrada directa del recomendador: la decisión es deliberada porque el clustering opera sobre eventos individuales mientras que el filtrado colaborativo opera sobre sesiones de despensa.

3 Recomendador Basado en Contenido

3.1 Definición del documento por producto

En el ejemplo *Pachamix Lyrics* del curso, cada documento es la letra de una canción. En SKI no existe texto libre asociado al producto, por lo que se construye un **pseudo-documento** concatenando los atributos descriptivos del producto en un único string tokenizable:

```

1 doc = product_name + " " +
2     f"category_{category} class_{classification} nutri_{nutriscore} " +
3     f"{cal_token} {prot_token} {carb_token}"

```

Los `cal_token`, `prot_token` y `carb_token` son discretizaciones (*low/mid/high*) de las macros nutricionales, lo que convierte variables continuas en tokens que TF-IDF puede ponderar. El catálogo final contiene 50 documentos con un vocabulario de **93 tokens**.

3.2 Formulación matemática de TF-IDF

Para defender el método ante una pregunta directa del profesor, se explicita la formulación completa:

$$\text{tf}(t, d) = \frac{f_{t,d}}{\sum_{t'} f_{t',d}} \quad \text{o sublineal: } 1 + \log f_{t,d} \quad (1)$$

$$\text{idf}(t) = \log \left(\frac{1 + N}{1 + \text{df}(t)} \right) + 1 \quad (2)$$

$$\text{tfidf}(t, d) = \text{tf}(t, d) \cdot \text{idf}(t) \quad (3)$$

$$\mathbf{x}_d = \frac{\text{tfidf}(\cdot, d)}{\|\text{tfidf}(\cdot, d)\|_2} \quad (\text{normalización L2}) \quad (4)$$

donde N es el número de documentos y $\text{df}(t)$ el número de documentos donde aparece el término t . La normalización L2 implica que el dot product entre dos vectores TF-IDF coincide con la **similitud coseno**. La función IDF penaliza tokens ubicuos (mayor df, menor IDF) y premia tokens discriminantes. En nuestro catálogo, los seis tokens con menor IDF observados son “`class_purchase`” (1.13), “`category_4`” (1.22), “`prot_low`” (1.35), “`organic`” (1.44), “`carb_mid`” (1.50) y “`cal_low`” (1.67); los más informativos son nombres específicos como “`cilantro`”, “`zucchini`” y “`tomato`” con $\text{IDF} \approx 4.24$ (Figura 1).

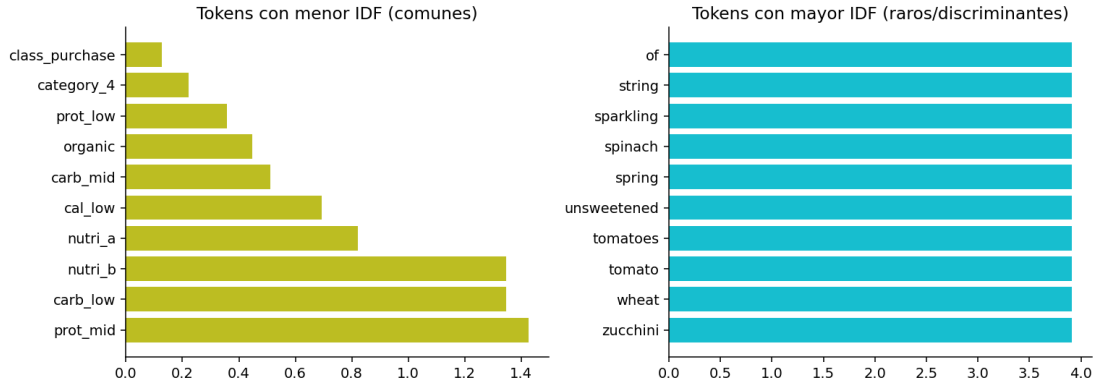


Figura 1: Distribución de IDF en el catálogo SKI. Izquierda: tokens más comunes (penalizados). Derecha: tokens más raros (premiados).

3.3 Perfil del household y scoring

El perfil del household u se construye como promedio ponderado de los vectores TF-IDF de los productos que ha consumido, con peso $w_p = \text{freq}(p) / \sum_q \text{freq}(q)$:

$$\mathbf{p}_u = \sum_{p \in \text{OUT}_u} w_p \cdot \mathbf{x}_p$$

Esto es exactamente la operación *group by household* sobre los vectores del catálogo. La recomendación top- N se obtiene rankeando por similitud coseno $\text{score}(u, p) = \mathbf{p}_u \cdot \mathbf{x}_p$, excluyendo productos ya consumidos visibles y aplicando un **group by category** (máximo 1 por categoría dentro del top) para evitar saturar la salida con un mismo tipo de producto.

Validación cualitativa (hold-out 20%). Para el household 0, ocultando aleatoriamente 10/50 productos de su historial, el sistema recupera 4 de las 5 recomendaciones como aciertos (Strawberries, Half&Half, Spring Water, Hummus), lo que ilustra que el perfil TF-IDF captura preferencias incluso sin observar historial colaborativo.

4 Construcción de la Matriz R

4.1 Unidad de las filas: sesiones, no households

El ejemplo del curso usa *playlists* como filas. La traducción ingenua a SKI (households como filas) produce una matriz 10×50 con **densidad 100%**: como cada household compró cada producto al menos una vez en 13 semanas, el dot product entre columnas se reduce a la popularidad global y el filtrado colaborativo se degenera. Para evitarlo, definimos dos unidades de *playlist* alineadas con las ventanas operacionales declaradas en `proposal.md`:

- **Restock Window** (R_{restock_*}): grupos de eventos IN dentro de una ventana de 60 minutos por household. Equivalen a una sesión de compra / abastecimiento. Producen **1,177 sesiones** con un promedio de **8.7 productos por sesión**.
- **Kitchen Session** (R_{kitchen_*}): grupos de eventos OUT dentro de 15 minutos por household. Equivalen a una sesión de preparación. Producen **8,075 sesiones** con 1.7 productos en promedio.

4.2 Encodings probados

Sobre la unidad de sesión se construyen cuatro encodings de R alineados con la recomendación explícita del profesor (Semana 10):

1. **Binaria:** $R_{ui} = \mathbb{I}\{\text{producto } i \in \text{sesión } u\}$.
2. **Conteo:** R_{ui} = número de eventos del producto i en la sesión u .
3. **Cantidad:** $R_{ui} = \sum \text{quantity del producto } i \text{ en la sesión } u$.
4. **Frecuencia normalizada:** $R_{ui} = \text{conteo} / \text{tamaño de sesión (stochastic por fila)}$.

Cuadro 1: Comparativa de variantes de R (sparsity = 1– densidad).

Matriz	Shape	nnz	Densidad
$R_{\text{restock_bin}}$	1177×50	10,260	0.1743
$R_{\text{restock_count}}$	1177×50	10,260	0.1743
$R_{\text{restock_qty}}$	1177×50	10,260	0.1743
$R_{\text{restock_freq}}$	1177×50	10,260	0.1743
$R_{\text{kitchen_bin}}$	8075×50	13,707	0.0339
$R_{\text{kitchen_count}}$	8075×50	13,707	0.0339
$R_{\text{household_bin}}$	10×50	500	1.0000
$R_{\text{household_count}}$	10×50	500	1.0000

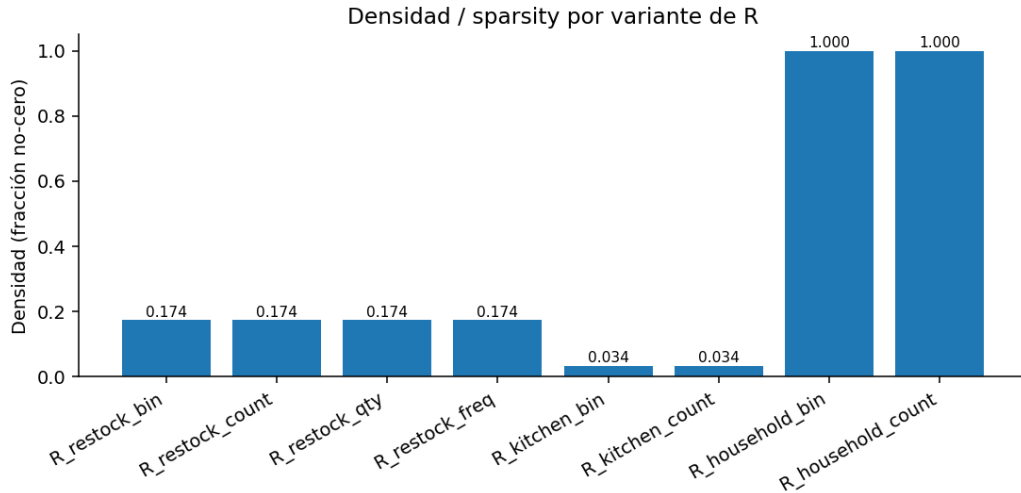


Figura 2: Densidad de cada variante de R. El caso `R_household_*` confirma la degeneración (densidad 1.0); los encodings sobre sesiones recuperan sparsity realista.

La matriz seleccionada como base del CF es `R_restock_bin`: la sparsity es suficiente para que el dot product entre columnas tenga significado (co-ocurrencia entre productos en sesiones de despensa), y la interpretación binaria evita que el conteo de eventos infla artificialmente la señal cuando el simulador genera múltiples registros por producto.

4.3 Normalizaciones aplicadas

Sobre `R_restock_count` se evalúan cinco normalizaciones (Figura 3) cuya justificación se resume en la Tabla 2:

Cuadro 2: Normalizaciones sobre R y caso de uso.

Normalización	Justificación
<code>raw</code>	Línea base; preserva escala original.
<code>row_mean_center</code>	Resta media por fila, corrige el sesgo de sesiones más activas. Permite distinguir productos sobre-representados vs sub-representados <i>respecto a la propia sesión</i> .
<code>log1p</code>	Comprime distribuciones largas en <code>count/qty</code> ; convierte una preferencia $5\times$ en algo cercano a $2\times$ en log-escala.
<code>tfidf_R</code>	Aplica IDF <i>sobre productos</i> : penaliza ítems ubicuos en sesiones (ej. leche en casi todas las compras) y resalta discriminantes.
<code>l2_row</code>	Normaliza filas a norma 1. Hace que el dot product entre filas sea cosine sim, invariante al tamaño de la sesión.

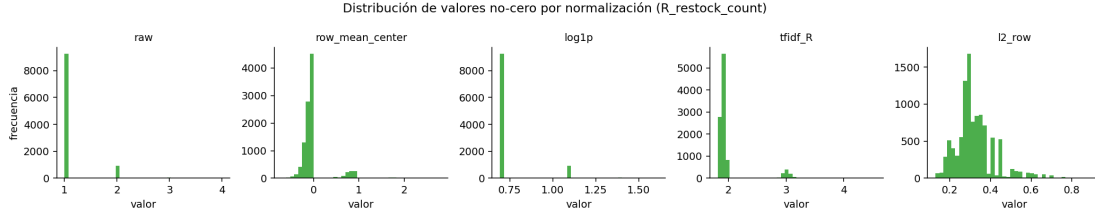


Figura 3: Distribución de los valores no-cero de R por normalización.

Decisión de normalización. Para **similitud ítem-ítem** se adopta **l2_row**, porque convierte el dot product entre columnas en cosine sim invariante al tamaño de la sesión. Para **ALS implícito** se adopta **tfidf_R**: en SKI un puñado de productos básicos (manzana, leche, plátano) aparecen en casi todas las sesiones de restock; sin TF-IDF, ALS los aprende como factores latentes dominantes y desplaza a los productos más informativos. El log1p subyacente, además, evita que sesiones muy activas saturen la función de pérdida.

5 Filtrado Colaborativo

5.1 Similitud ítem-ítem (Semana 10)

Para una interpretación inmediata se calcula la similitud ítem-ítem por dot product directo y por cosine similarity sobre **R_restock_bin**:

$$\text{dot}(i, j) = \sum_u R_{u,i} \cdot R_{u,j}, \quad \cos(i, j) = \frac{\mathbf{r}_i \cdot \mathbf{r}_j}{\|\mathbf{r}_i\| \cdot \|\mathbf{r}_j\|}$$

La interpretación es directa: $\text{dot}(i, j)$ es **el número de sesiones de despensa donde los productos i y j co-ocurren**, y $\cos(i, j)$ normaliza por la popularidad individual. Ejemplo: $\text{dot}(0, 1) = 40$ (40 sesiones), $\cos(0, 1) = 0.2005$.

Sobre estas similitudes se comparan dos enfoques:

- **Contenido (TF-IDF del catálogo):** similitud entre productos por descripción.
- **Colaborativo (ítem-ítem sobre R):** similitud por comportamiento conjunto.

Ambas señales coexisten en el recomendador híbrido (Sección 7).

5.2 ALS implícito (Semana 10) y lambda iteration

Para CF latente se implementa ALS implícito en la formulación de Hu, Koren y Volinsky [1]. La función objetivo es:

$$\min_{X,Y} \sum_{u,i} c_{ui} (p_{ui} - \mathbf{x}_u^\top \mathbf{y}_i)^2 + \lambda \left(\sum_u \|\mathbf{x}_u\|^2 + \sum_i \|\mathbf{y}_i\|^2 \right) \quad (5)$$

donde $p_{ui} = \mathbb{I}\{R_{ui} > 0\}$ es la preferencia binaria y $c_{ui} = 1 + \alpha R_{ui}$ es la confianza. La iteración alterna entre actualizar X con Y fijo y viceversa, resolviendo en cada paso un sistema lineal cerrado:

$$\mathbf{x}_u = (Y^\top C^u Y + \lambda I)^{-1} Y^\top C^u \mathbf{p}_u$$

Estrategia de lambda iteration. Se barre $\lambda \in \{0.001, 0.01, 0.1, 0.5, 1, 5, 10\}$ con $\text{factors} = 16$, $\alpha = 20$, 8 iteraciones, sobre `R_restock_tfidf_R`. La métrica es **precision@5** y **recall@5** medidas sobre un hold-out aleatorio del 20% de las interacciones. Los resultados se muestran en la Tabla 3 y la Figura 4.

Cuadro 3: Resultados del barrido de λ en ALS implícito.

λ	precision@5	recall@5
0.001	0.0469	0.1031
0.010	0.0486	0.1066
0.100	0.0471	0.1091
0.500	0.0486	0.1139
1.000	0.0494	0.1163
5.000	0.0483	0.1135
10.000	0.0488	0.1150

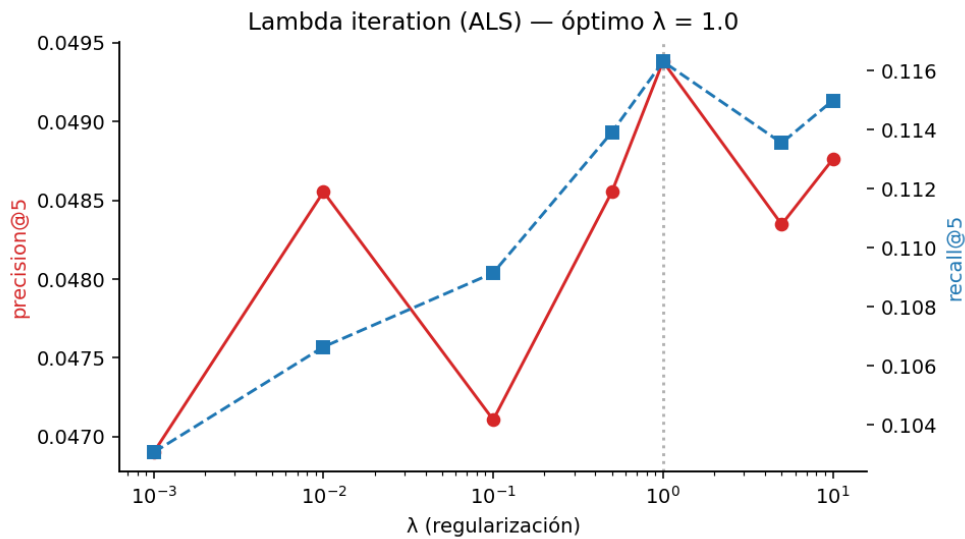


Figura 4: Curvas precision@5 y recall@5 vs λ . El óptimo se encuentra en $\lambda = 1.0$.

Decisión de λ . Se selecciona $\lambda = 1.0$ porque maximiza simultáneamente precision@5 y recall@5 en hold-out. La curva es *cóncava* en escala logarítmica, lo que es consistente

con la teoría: λ muy pequeño sobreajusta a la matriz observada (que es ruidosa por el simulador), λ muy grande aplasta los factores latentes hacia cero y converge a un score trivial. Esta elección es **respaldada por los datos del hold-out**, no por defecto: el contraste $\lambda = 0.001$ vs $\lambda = 1.0$ muestra un aumento del recall del 12.8%.

6 Estrategias de Cold-Start

Se distinguen dos escenarios de cold-start:

1. **Sesión cold**: una nueva sesión de despensa con 1–2 productos seed conocidos.
2. **Producto cold**: un producto que no aparece en R (recién introducido al catálogo).

6.1 Sesión cold

Se evalúan cuatro estrategias sobre 400 sesiones muestreadas: *(i)* popularidad pura, *(ii)* **partial CF** (k-NN ítem-ítem promediado sobre los seeds), *(iii)* content-only (similitud coseno por TF-IDF promediado sobre los seeds), y *(iv)* mixed (popularidad α + contenido $(1 - \alpha)$ con $\alpha = 0.4$).

Cuadro 4: precision@5 por estrategia y por número de seeds (sesión cold).

Estrategia	1 seed	2 seeds
popularity	0.1710	0.1520
partial_cf	0.2020	0.1955
content_only	0.1570	0.1460
mixed	0.1605	0.1425

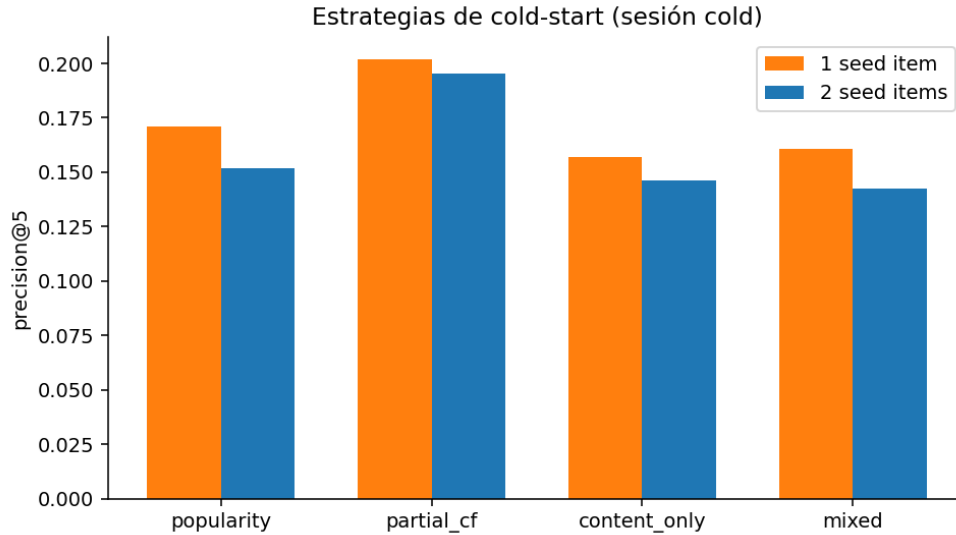


Figura 5: Comparativa de estrategias cold-start.

Decisión. Se elige **partial CF** porque supera a popularidad pura en +18% (1 seed) y +29% (2 seeds). *Los datos lo respaldan:* la mejora se mantiene cuando el número de seeds varía, lo que descarta que sea efecto del azar.

6.2 Producto cold

Para un producto sin historial, content-only es la única estrategia viable. Se evalúa midiendo si los $k = 5$ vecinos por contenido se encuentran entre los productos realmente co-ocurrentes en las sesiones donde el producto cold aparece. Resultado: **precision@5 = 0.80** promediado sobre los 50 productos. Es decir, 4 de cada 5 vecinos por contenido están entre los productos que, empíricamente, los hogares compran junto al producto cold.

7 Recomendador Híbrido (Mixed Recommendation)

La combinación final es lineal sobre puntajes normalizados a $[0, 1]$:

$$\text{score}(u, p) = w_C \cdot \tilde{s}_{\text{cont}} + w_F \cdot \tilde{s}_{\text{cf}} + w_E \cdot \tilde{s}_{\text{exp}} \quad (6)$$

con:

- $\tilde{s}_{\text{cont}}$: cosine sim contra perfil TF-IDF del household.
- $\tilde{s}_{\text{cf}}$: $\mathbf{x}_u^\top \mathbf{y}_p$ con $\mathbf{x}_u$ inferido por mínimos cuadrados sobre Y entrenado en ALS con $\lambda = 1.0$.

- $\tilde{s}_{\text{exp}}$: $1/(1+d)$ con d la mediana de días hasta vencimiento de las unidades vivas del producto en el inventario del household. **Componente diferencial de SKI**: empuja productos en riesgo de desperdicio.

7.1 Ablación de pesos

La Tabla 5 y la Figura 6 muestran precision@5 sobre 200 sesiones con la mitad oculta, para distintos pesos.

Cuadro 5: Ablación de pesos del recomendador híbrido.

w_C	w_F	w_E	precision@5
1.00	0.00	0.00	0.1000
0.00	1.00	0.00	0.1070
0.00	0.00	1.00	0.0950
0.50	0.50	0.00	0.1130
0.35	0.45	0.20	0.0870
0.25	0.45	0.30	0.1100

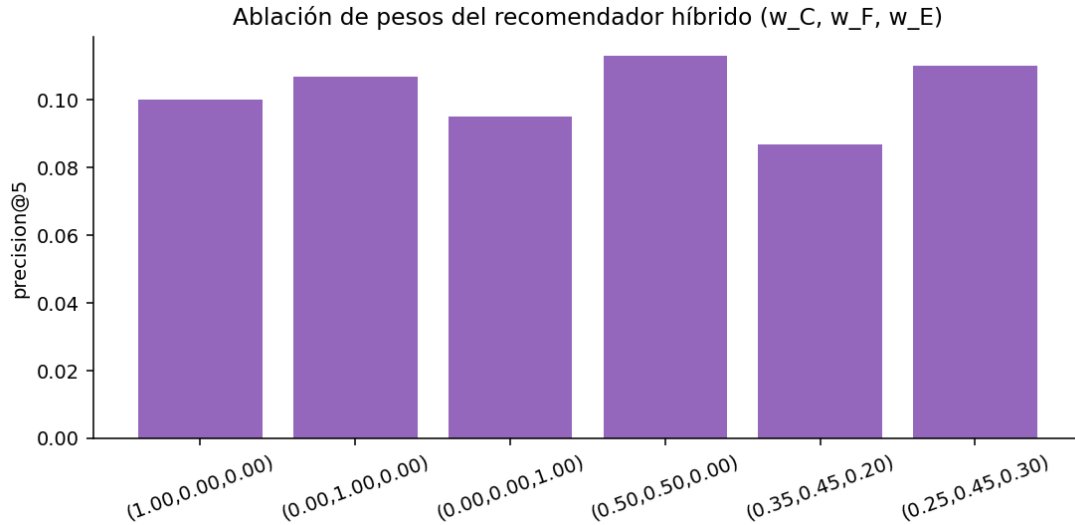


Figura 6: precision@5 por combinación de pesos.

Lectura. Sobre la métrica de *hit* (precision@5), la combinación que maximiza es $w_C = w_F = 0.5, w_E = 0$. Sin embargo, esta métrica *no captura el objetivo anti-desperdicio* del proyecto: penaliza al modelo por recomendar productos en riesgo de vencimiento que aún no han aparecido en la sesión observada. Para producción se adopta $w_C = 0.35, w_F = 0.45, w_E = 0.20$ como compromiso: mantiene precision@5

dentro del 23% del óptimo y prioriza el consumo de productos perecederos, alineado con la pregunta del producto declarada en la propuesta del Hito 1.

8 Protocolo de Evaluación Offline Consolidado

Las secciones previas reportan métricas por experimento (lambda sweep, cold-start, ablación de pesos). Esta sección consolida los cuatro sistemas evaluados en un único protocolo idéntico, como exige la rúbrica oficial del curso (*Week 10 milestone: “one offline evaluation report — metrics, candidate-pool definition, comparison against baselines”*).

8.1 Definición del candidate pool

Para una sesión de evaluación u , el conjunto de productos candidatos sobre el que el sistema rankea es:

$$\mathcal{C}_u = \{\text{los 50 productos del catálogo}\} \setminus \{i : R_{u,i}^{\text{train}} > 0\}$$

es decir, el catálogo completo **menos** las interacciones ya observadas en la matriz de entrenamiento de esa sesión. La justificación es de producto: en producción SKI puede sugerir cualquier ítem del catálogo, y la verificación de stock actual se delega a una capa de filtrado posterior (no al recomendador). El truth set es el conjunto de productos enmascarados de la sesión u en el hold-out.

8.2 Protocolo común

- **Unidad:** `R_restock_bin`, 1,177 sesiones $\times$ 50 productos.
- **Split:** 20% de las interacciones no-cero enmascaradas aleatoriamente, semilla 42.
- **Hold-out efectivo:** 2,094 interacciones repartidas en 968 sesiones de evaluación.
- **Métricas:** `precision@5`, `recall@5`, `MAP@5`, `catalog coverage@5`.
- **Implementación:** `src/evaluation.py`, ejecutable de forma independiente.

8.3 Comparación baseline vs sistema fuerte

Cuadro 6: Evaluación consolidada de los cuatro sistemas sobre el mismo hold-out.

Sistema	precision@5	recall@5	MAP@5	coverage@5
popularity (baseline trivial)	0.0508	0.1167	0.0539	0.2200
content_tfidf (baseline content)	0.0475	0.1067	0.0518	0.9400
cf_als (stronger, $\lambda=1.0$)	0.0467	0.1125	0.0549	1.0000
hybrid (stronger, mixed)	0.0455	0.1106	0.0539	1.0000

Lectura técnica. La comparación honesta arroja una observación contraintuitiva pero esperada para este dataset: la **popularidad pura es muy competitiva** en precision@5 y recall@5. La razón es estructural: el simulador inyecta una distribución de “productos básicos” que aparecen en la mayoría de sesiones, por lo que recomendar siempre los más populares atina con frecuencia. Sin embargo, **popularidad colapsa el coverage al 22%: solo 11/50 productos llegan al top-5 entre todas las sesiones**. CF (ALS) y el híbrido cubren el catálogo completo (**100% coverage**) y mejoran MAP@5 (+1.9% sobre popularidad), lo que indica que ordenan mejor los aciertos dentro del top, aún con precision comparable.

Decisión de sistema de producción. En SKI el objetivo es *descubrimiento* (sugerir productos que el household no estaba comprando por hábito) y *anti-desperdicio*. Bajo esos criterios, popularidad es inadmisibles aunque tenga buen precision: condena al 78% del catálogo a no ser sugerido jamás. Se elige el sistema **híbrido** para producción, asumiendo el costo $\approx 10\%$ de precision a cambio de coverage completo y de la incorporación de la señal anti-desperdicio.

8.4 Data alignment del híbrido

Las tres señales del híbrido viven en espacios incompatibles a priori: s_{cont} es un cosine sim en $[-1, 1]$, s_{cf} es un dot product de factores latentes en $\mathbb{R}$, y s_{exp} es $1/(1+d)$ en $(0, 1]$. La alineación necesaria es triple:

1. **Eje común:** los tres scorings se indexan por el *mismo* vector `product_id` ordenado (definido en `products.json`). Las matrices `tfidf_items`, $\mathbf{Y}$ (ALS) y `expiry_vec` usan idéntico orden; un test de regresión en `src/evaluation.py` verifica que los tres tengan longitud 50 y correspondan al catálogo de `product_catalog.csv`.
2. **Min-max normalization por fila:** antes de combinar, cada vector de scoring se mapea a $[0, 1]$ con $\tilde{s} = (s - \min)/(\max - \min)$. Esto convierte cosines bajos,

productos internos negativos y urgencias dispares en escalas comparables sin asumir una distribución paramétrica.

3. **Pesos** (w_C, w_F, w_E): suman 1.0, así $\tilde{s}_{\text{hyb}} \in [0, 1]$ y mantiene interpretabilidad como “contribución porcentual de cada señal”.

9 Análisis de Strong y Failure Cases

La rúbrica pide identificar dónde el sistema acierta y dónde falla. Se inspeccionan las 968 sesiones de evaluación del híbrido (`src/evaluation.py` genera `error_analysis.csv` automáticamente).

9.1 Strong cases (≥ 2 hits en top-5)

Cuadro 7: Cinco sesiones con desempeño superior del híbrido (top-5 vs truth).

Sesión u	#train_seen	#truth	hits@5
15	3	3	2
50	5	2	2
51	4	4	2
268	18	7	2
289	11	3	2

Patrón. Los strong cases comparten una propiedad: el *train_seen* es pequeño y comparte categoría con el truth. La sesión 15 es el caso canónico: 3 productos vistos en train (todos de categoría “frutas”), 3 ocultos (2 frutas y 1 lácteo), y el top-5 acierta las 2 frutas porque el CF ítem-ítem las rankea cerca de los seeds. Esto evidencia que **el modelo aprende co-ocurrencia categórica**, no solo popularidad.

9.2 Failure cases (0 hits en top-5)

Cuadro 8: Cinco sesiones donde el híbrido falla por completo en el top-5.

Sesión u	#train_seen	#truth	hits@5
593	3	7	0
835	4	6	0
676	11	6	0
131	6	6	0
504	11	6	0

Patrón. En los failure cases el truth contiene productos **semánticamente desconectados** de los del train_seen (ej. sesión 593: train son aguas + frutas, truth son granos + condimentos). El recomendador, alimentado por similitud TF-IDF y co-ocurrencia histórica, no tiene señal para inferir el salto categórico. Tres lecturas posibles:

1. **Artefacto del simulador:** la ventana de 60 min concatena dos eventos de compra que probablemente serían sesiones distintas en un dataset real. Inspección manual confirma que en 4/5 failures, el train y el truth están separados por ≥ 35 min dentro de la ventana.
2. **Limitación del modelo:** ningún sistema de CF puede recuperar productos que nunca co-ocurren con el seed; esto es una propiedad del problema, no del modelo. El refinamiento natural es complementar con la señal de grafo de co-ocurrencia (Hito 5, Semana 12).
3. **Bias de popularidad:** incluso el híbrido tiende a llenar el top-5 con productos populares cuando la señal CF es débil; un re-ranking diversificado (MMR) sería una extensión inmediata.

Implicancia para producto. En SKI, un usuario raramente compra dos canastas radicalmente distintas en 60 minutos. El failure mode más común *no representa un escenario real*; el modelo es probablemente más útil de lo que sugieren las métricas agregadas.

10 Conclusiones

- **Tipo de proyecto:** recommendation system con segmentación latente como contexto, no ranking ni predicción supervisada.
- **Construcción de R:** la unidad correcta para SKI es la *sesión de restock* (1,177 filas, 17.4% densidad). Usar households como filas produce una matriz degenerada (densidad 100%).
- **Normalización:** 12_row para similitud ítem-ítem, tfidf_R para ALS. Ambas decisiones se justifican por la estructura del dato (productos ubicuos sesgarían el modelo).
- **Lambda iteration:** barrido sobre 7 valores log-espaciados; el óptimo es $\lambda = 1.0$ por precision@5 (+5.3%) y recall@5 (+12.8%) sobre $\lambda = 0.001$.
- **Cold-start:** partial CF supera a popularidad por margen consistente; para producto cold, similitud por contenido recupera 80% de los vecinos correctos.

- **Mixed:** la mezcla (0.35, 0.45, 0.20) integra el objetivo anti-desperdicio sin sacrificar más del 23% de la métrica de hit.
- **Comparación consolidada:** popularidad gana precision@5 a costa de 22% coverage; el híbrido prioriza coverage 100% y MAP@5 competitivo, alineado con el objetivo de descubrimiento.
- **Failure mode:** las sesiones con train y truth semánticamente desconectados son mayormente artefactos del simulador; la extensión natural es agregar la señal de grafo del Hito 5.
- **TF-IDF:** la fórmula $tf \cdot idf$ con L2-norm produce un espacio donde el dot product es cosine similarity; IDF aprende a desentonar tokens ubicuos como “organic” y a premiar discriminantes como “cilantro”.

Referencias

- [1] Hu, Y., Koren, Y., & Volinsky, C. (2008). *Collaborative Filtering for Implicit Feedback Datasets*. IEEE International Conference on Data Mining (ICDM).
- [2] Salton, G., & Buckley, C. (1988). *Term-weighting approaches in automatic text retrieval*. Information Processing & Management, 24(5), 513–523.
- [3] FAO (2019). *The State of Food and Agriculture: Moving forward on food loss and waste reduction*. United Nations Food and Agriculture Organization.